

コードの時系列変化を考慮した保守性低下 の要因分析と改善

鈴木研究室 笹川 尋翔

背景1: ソフトウェア開発の課題

持続的なソフトウェア開発の難しさ

- ソフトウェアは時間とともに複雑化、保守が困難に
- 保守性はソフトウェア品質の維持に不可欠

リファクタリングの複雑さ

- 76%の開発者がリファクタリングによるバグ混入やリグレッションのリスクを認識 [1]

背景2: リファクタリング手法の限界

静的解析ツールを用いた品質評価の普及

- 構文チェックやコードメトリクスの異常検出が自動テストに含まれることも

時系列的な変化の不足

- 単一時点での分析に留まっている
- クラスや関数の状態が変化しても追跡できない

目的

- 時系列変化量を用いたバグ予測精度の向上
 - 保守性メトリクス（例: コード行数、循環的複雑度）の変化量を導入・分析
- バグの特徴と保守性メトリクスの関係性の解明
 - 欠陥分析により、バグの特徴を体系的に整理
 - 保守性とのつながりを指摘し、具体的な改善策を提示

関連研究

- レビューコメントによるコードスメルの分析^[2]
 - レビューアーは保守性の低下要因をコーディング規約から特定
- 静的解析ツールによるリファクタリング支援^[3]
 - 静的解析ツールを使用すると...
 - リファクタリングの機会が増えるが、問題の背後にある理論的な根拠を理解するのが難しい

[2] X.Han et al., "Understanding Code Smell Detection via Code Review: A Study of the OpenStack Community"

[3] S.Romano et al., "Do Static Analysis Tools Affect Software Quality when Using Test-driven Development?"

手順1: データ分析基盤の開発

- データセット
 - BugHunter Dataset^[4]を使用
- データ分析の流れ
 1. 欠損値の削除
 2. 新しいコードメトリクスの計算
 3. 特徴量変換
 4. 機械学習モデルの学習・評価
 5. モデルの性能に影響する特徴量の可視化

手順2: 特徴量エンジニアリング

- 新しい特徴量
 - コード行数の変化量
 - トークン数の変化量
 - 循環的複雑度の変化量
 - メソッド操作の種類（例: 追加、削除、変更）
- 特徴量変換
 - テキストデータ: TF-IDF
 - カテゴリカル変数: One-Hotエンコーディング

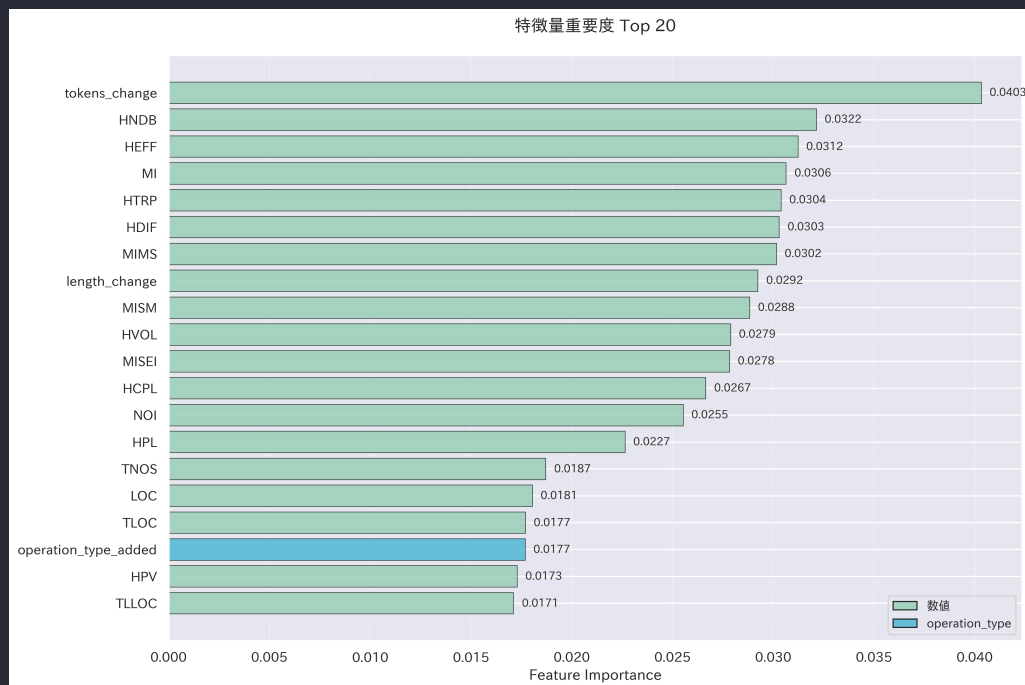
実験: 6プロジェクトの分析

- BugHunter Datasetから6プロジェクトを選定
- 各プロジェクトで最大3000件のデータを使用
- 対象プロジェクト
 - antlr4: 構文解析ツール
 - BroadleafCommerce: Webアプリフレームワーク
 - ceylon-ide-eclipse: IDEプラグイン
 - elasticsearch: 全文検索エンジン
 - hazelcast: インメモリデータベース
 - oryx: 機械学習フレームワーク

実験: 6プロジェクトの分析

- 分析手法
 - 特徴量重要度の分析
 - ヒストグラムによる特徴量分布の比較
 - Partial Dependence Plot (PDP) の分析
 - 決定木の可視化による分類条件の抽出

結果1: 特徴量重要度



- トークン数・コード行数の変化量の重要度が高い
- Halsteadメトリクス、Maintainability Index (MI) も上位に位置している

図1: hazelcastの特徴量重要度

結果2: 特徴量分布

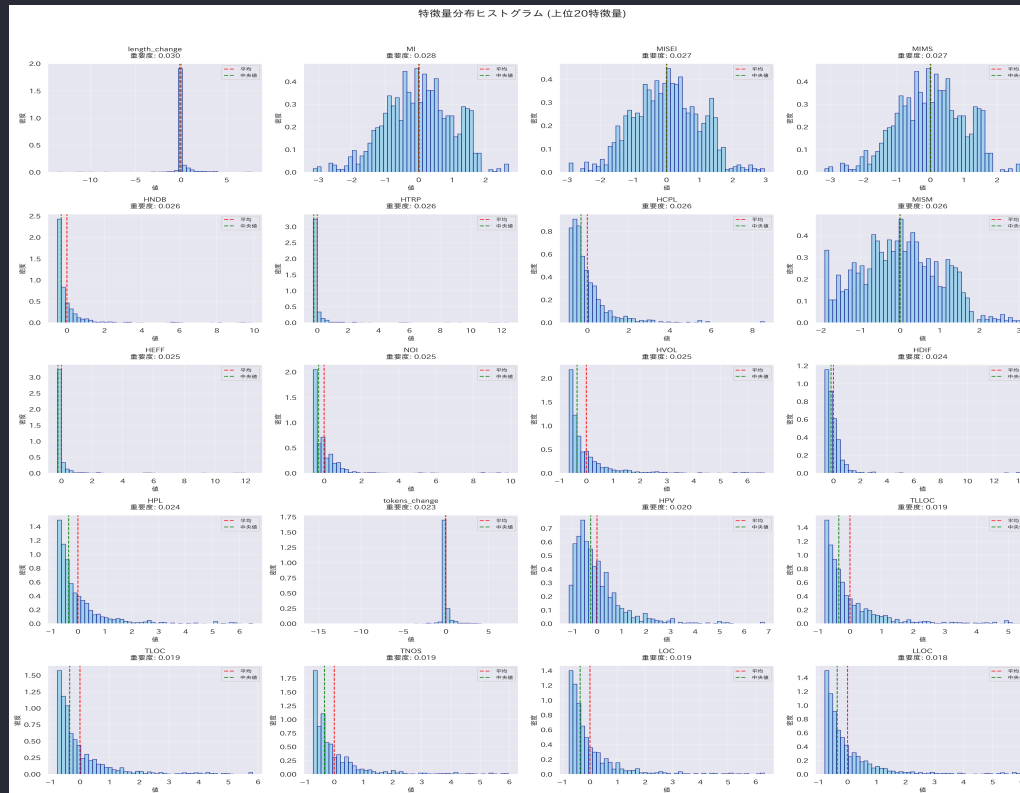


図3: ceylon-ide-eclipseの特徴量分布

- 変化量・Halsteadメトリクスは分散が小さく、特定の箇所に値が集中
- MIは分散が大きい
- 一峰性の分布が多い

結果3: PDP分析

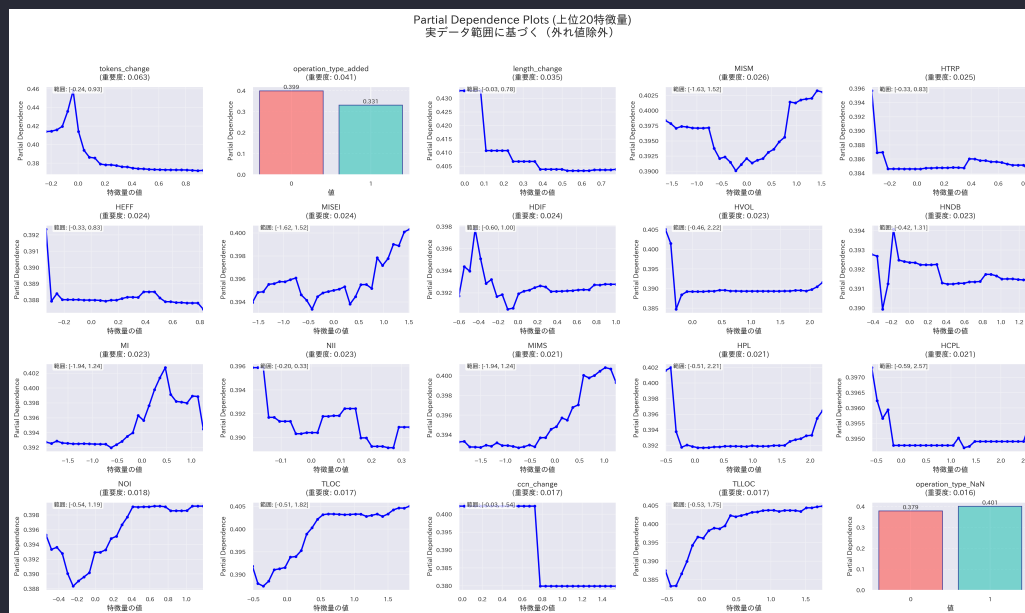


図3: elasticsearchのPDP

- トークン数・コード行数の変化量が0付近のとき: 陽性クラスの予測確率が上昇
- ほとんどの特徴量において、陽性クラスの予測確率が0.5未満

結果4: 決定木分析

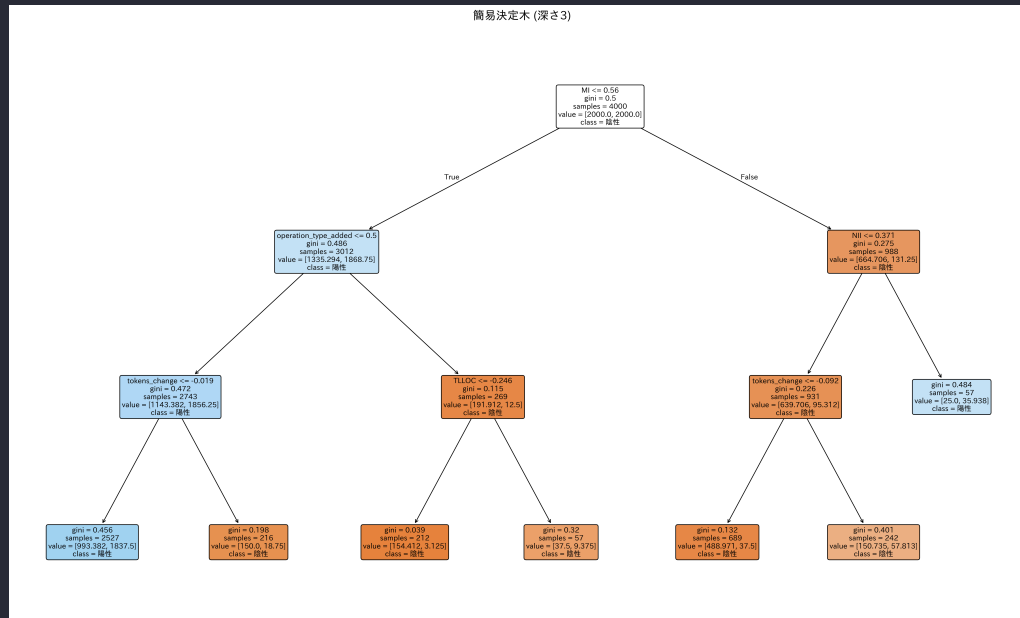


図3: elasticsearchの決定木

- 陰性クラスのノードのジニ不純度が比較的低い
- 高い確信度で陰性クラスを分類している

結果5: モデルの評価指標

- 3プロジェクト: F1スコアが最大0.1向上、有意差あり
- 2プロジェクト: F1スコアが向上、有意差なし
- 1プロジェクト: F1スコアが低下、有意差なし

評価・考察

陰性クラス予測の改善

- リファクタリングによるメトリクスへの減少傾向を確認できた
- 消去法的な分類がF1スコアの改善に寄与

陽性クラス予測の曖昧さ

- PDPや決定木を見ると、陽性クラスの予測確率が低い
- バグ混入理由が多様であることが影響している？

課題・展望

陽性クラスの詳細な分類による因果関係の解明

- バグ混入と保守性メトリクスに関連性が不明確

バグの体系的分類によるパターンの識別

- Orthogonal Defect Classification (ODC) のような欠陥分析
- バグの特徴とコードメトリクスの時系列変化の関連性を示す

付録1: 保守性とは

- JIS X 0129による定義
 - 修正のしやすさに関するソフトウェア製品的能力
 - 修正: 「是正」、「向上」、「環境の変化、要求仕様や機能仕様の変更」にソフトウェアを適応させること」
- 保守性メトリクスの例
 - 複雑度: 循環的複雑度、ネストの深さ
 - 結合度: ファンイン/ファンアウト
 - 凝集度: ある変数を参照しているメソッドの割合
 - サイズ: コード行数、メソッド数

付録2: BugHunter Datasetについて

- 特定のファイル、クラス、メソッドのデータセット
- バグ混入コミット（陽性）とバグ修正コミット（陰性）を含む
- コミットごとのコードメトリクスを計算・記録
- コミットIDと識別子を使用してデータセットを拡張できる

付録3: 機械学習手法について

- モデル: ランダムフォレストを使用
 - 先行研究で最もバグの有無の予測精度が高かったため
- 前処理: アンダーサンプリングを使用
 - 先行研究と同じ
- 交差検証の回数: 10回
 - 先行研究と同じ
- その他の処理: 独自に実装
 - プログラムが非公開であるため